第三阶段：基线与延迟加速分析

工大喵喵队 2026 年 11 月

一、基线实现

基线用标准 RVV 的 `vwmul.vv + vwredsum.vs`，不是手写标量循环。

`software/kernels.S` 里的 `vdot_baseline`：

```
vdot_baseline:
    vsetivli zero, 4, e32, m1, tu, mu
    vmv.v.i  v3, 0
    vmv.v.i  v6, 0
    vsetivli zero, 8, e8, m1, tu, mu
    vle8.v   v1, (a0)
    vle8.v   v2, (a1)
    vwmul.vv v4, v1, v2
    vsetivli zero, 8, e16, m2, tu, mu
    vwredsum.vs v3, v4, v6
    vsetivli zero, 4, e32, m1, tu, mu
    vse32.v  v3, (a2)
    ret
```

为什么这样写：

· `vwmul.vv` 把 8 个 int8 相乘，结果扩宽到 16 位（EEW=16，EMUL=2），避免 8 位乘积溢出；

· `vwredsum.vs` 把 8 个 16 位乘积做扩宽归约，累加到 32 位 `v3`；

· 归约前把 `v3`、`v6` 清零，否则初始值会影响结果；

· 不能用普通 `vredsum`，因为 16 位归约会溢出：$8\times(-128\times-128)=131{,}072 > 32{,}767$。

自定义实现：

```
vdot_custom:
    vsetivli zero, 8, e8, m1, tu, mu
    vle8.v   v1, (a0)
    vle8.v   v2, (a1)
    .word    0x0220A18B      /* vdot.vv v3,v1,v2 */
    vsetivli zero, 4, e32, m1, tu, mu
    vse32.v  v3, (a2)
    ret
```

vdot 直接写 `v3` 低 32 位并清零高 96 位，不需要预先清零，也不需要 16 位归约。

静态指令数：

实现	指令数	组成
基线	12	4×vsetvli + 2×vmv + 2×vle8 + vwmul + vwredsum + vse32 + ret
vdot	7	2×vsetvli + 2×vle8 + vdot + vse32 + ret
差值	5	少 2×vsetvli、2×vmv、1 个计算指令

二、测量方法

`software/perf_main.c` 的步骤：

1.准备固定输入 `a[i]=i*7-60`、`b[i]=i*3-20`； 2.用软件 oracle 算期望值；3.先跑一遍 baseline、一遍 custom，检查 `out[0]==expected` 且 `out[1..3]==0`； 4.再各跑一遍预热； 5.用 `rdcycle` 计时，各跑 256 次； 6.通过 UART 打印结果。

核心代码：

```
start = cycles();
for (int i = 0; i < 256; i++) vdot_baseline(a, b, out);
baseline_cycles = cycles() - start;

start = cycles();
for (int i = 0; i < 256; i++) vdot_custom(a, b, out);
custom_cycles = cycles() - start;
```

两种顺序：

· `make perf`：先基线后 vdot；

· `make perf-reverse`：`-DPERF_REVERSE=1`，先 vdot 后基线。

计时包含 `vsetvli`、两条 `vle8`、计算、`vse32`、函数调用/返回和循环开销；不包含输入准备、正确性检查和结果打印。所以这是完整可调用内核的延迟，不是单条计算指令的延迟。

三、测量结果

顺序	基线周期	vdot 周期	加速比	周期减少
先基线后 vdot	7,881	5,141	1.532970×	34.77%
先 vdot 后基线	7,882	5,119	1.539754×	35.05%
平均	7,881.5	5,130	1.536×	34.91%

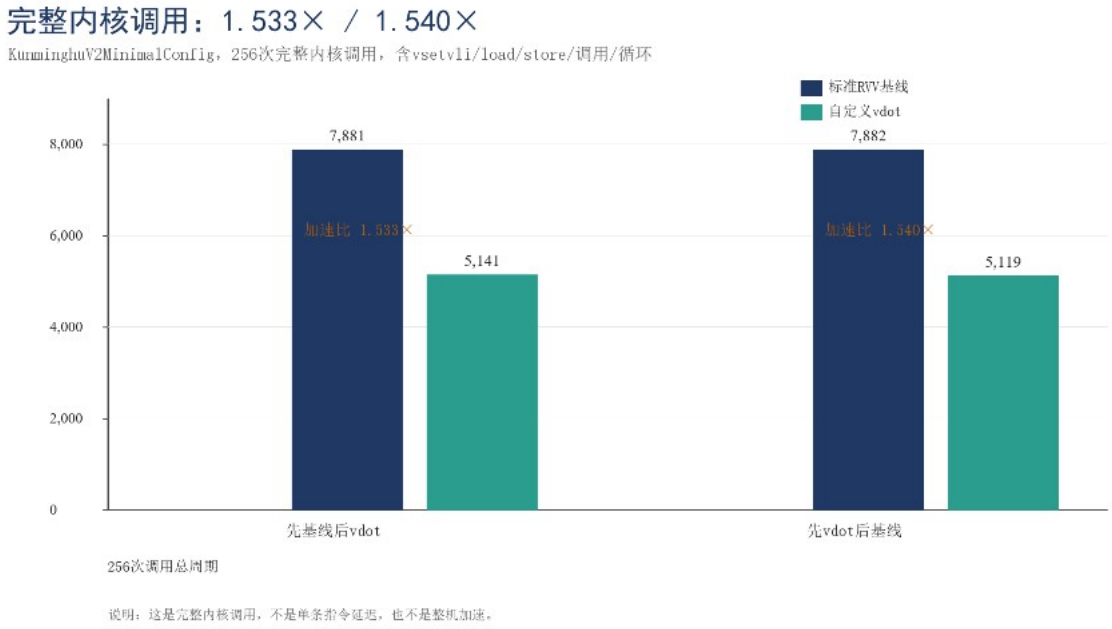


图 1 256 次内核调用的总周期对比

每次调用的平均周期：

实现	平均周期/调用
标准 RVV 基线	30.79
自定义 vdot	20.08
差值	10.70

原始日志：

```
PERF_ORDER baseline-first
PERF_V1 iterations=256 baseline_cycles=7881 custom_cycles=5141

PERF_ORDER custom-first
PERF_V1 iterations=256 baseline_cycles=7882 custom_cycles=5119
```

日志在 `results/kunminghu-v2-final/perf-baseline-first.log` 和 `perf-custom-first.log`。

四、两种测量结果：1.53× 与 12×

1.完整内核调用：1.53×

以先基线后 vdot 为例：

```
基线：7,881 / 256 = 30.79 周期/调用
vdot：5,141 / 256 = 20.08 周期/调用
节省：10.70 周期/调用
加速比：30.79 / 20.08 = 1.533×
```

这个 1.53× 是“一次完整函数调用”的周期比，包含固定开销，不是单条 vdot 指令的固定 latency，也不是整机加速或 LLM 收益。

为什么不是 8×：

· 一次调用里除了点积计算，还有 `vsetvli`、两条 `vle8`、`vse32`、函数调用和返回；

· 8 个元素的点积本身很小，固定开销占比高；

· 基线虽然指令多，但也是向量指令，不是标量循环，差距不会拉到 8 倍。

2.计算路径观测区间：12×

`results/latency-comparison/latency-evidence.json` 里还有一组 MinimalConfig 波形对照。统一定义是“首个算术微操作被功能单元接收 → 最终完整结果在功能单元输出有效且被接收”：

路径	输入接收周期	结果有效周期	观测区间
vdot	2390	2392	2 周期
标准 RVV 宽乘+宽归约	2408	2432	24 周期

观测区间比是 24 / 2 = 12×。它只表示这条计算路径的区间比，包含基线路径里的微操作依赖和调度等待；不含 `vsetvli`、访存、函数调用和循环，不是整机加速、不是吞吐率，也不能外推到 `KunminghuV2MinimalConfig`。

这两个数字不能混在一起说：1.53× 是完整内核调用，12× 是计算路径观测区间；一个来自 `KunminghuV2MinimalConfig`，一个来自 `MinimalConfig`。

五、顺序影响

指标	baseline-first	custom-first	差异
基线周期	7,881	7,882	+1（+0.013%）
vdot 周期	5,141	5,119	−22（−0.43%）
加速比	1.532970	1.539754	+0.44%

基线几乎不变，vdot 在 custom-first 下快 22 周期（0.43%），可能是缓存/预热/顺序效应。两个方向的加速比都落在 1.53–1.54×，结论对顺序不敏感。

没有做多次重复的均值和方差，不能给统计置信区间；22 周期的差异也不应该过度解释。

六、结论与限制

结论：

1.标准 RVV 基线功能正确，是公平的“已有指令组合”基线。 2.在 `KunminghuV2MinimalConfig` 下，256 次完整内核调用：先基线 7,881 → 5,141，1.532970×；先 vdot 7,882 → 5,119，1.539754×。 3.每次调用平均 30.79 → 20.08 周期，节省 10.70 周期；两个顺序结论一致。 4.vdot 数据通路 2 级；完整内核 1.53×、计算路径 2 vs 24 周期（12×）是两种测量，都不是整机加速。

限制：

· 仿真环境，不是上板实测；

· 固定输入、固定 256 次、单次测量，没有统计置信区间；

· 完整内核调用，含固定开销；没有单条指令隔离实验；

· 没有综合/STA，所以没有频率、面积、功耗、时序余量；

· 性能数据来自正式配置，波形来自 `MinimalConfig`，两个配置不能混；

· 12×只来自一组 MinimalConfig 波形，不是整机加速、不是吞吐率，不能外推；

· 参数化只到独立模块级（n=4/8/16），未接进香山，也没有综合/时序数据；

· 基线是标准 RVV 宽乘+宽归约，不是标量循环，也不是手写汇编最优实现。